Compte rendu

Travaux pratiques

Conteneurisation et orchestration de microservices

Élaboré par :

Farah Ayeb
Hanine Ben Amor
Emna Drihem
Yasmine Ferjani

RT5

Dépôt GitHub : `https://github.com/AyebFarah/DevOps`

Image Docker Hub : `https://hub.docker.com/r/ayebfarah/counter-app`

1 Introduction

Ce rapport présente notre travail sur Docker. On a déployé une application appelée Visit Counter. Elle compte les visites d'une page web.

L'application contient deux services. Le service web affiche le nombre de visites. Il affiche aussi l'identifiant du conteneur qui répond. Le service Redis stocke le compteur sur disque.

On a ajouté un load balancer Nginx pour la partie Docker Compose. On a aussi créé un pipeline Jenkins. Il teste le code, construit l'image et la publie sur Docker Hub.

1.1 Architecture

À chaque requête sur la page d'accueil, le service web contacte Redis. Redis incrémente la clé `hits`. Le service web affiche la valeur et son hostname. Le hostname correspond à l'identifiant court du conteneur.

Service	Rôle	Image	Port hôte
web	Application Flask et Gunicorn	counter-app :1.0	9080 à 9090
db-service	Base Redis avec persistance	redis :7-alpine	aucun
lb	Load balancer Nginx	nginx :alpine	8088

Table 1 – Les services du projet

Les trois services partagent le réseau `app-network`. Redis écrit ses données dans le volume `redis-data`.

2 Partie A : Apprentissage avec les videos

On a suivi le cours vidéo Packt. Il explique comment conteneuriser une application Spring Boot. Il montre aussi comment lancer MongoDB avec Docker Compose.

2.1 Notions retenues

— Une image est un modèle. Elle contient l'application et ses dépendances.
— Un conteneur est une image en cours d'exécution.
— Un réseau Docker permet aux conteneurs de se joindre par leur nom.
— Un volume garde les données en dehors du conteneur.
— Docker Compose décrit et lance plusieurs services ensemble.
— Docker Hub est le registre où On publie mes images.
— Jenkins exécute les étapes d'un pipeline de façon automatique.

subsectionManipulations faites

On a construit l'image `application-one-img` avec l'application Spring Boot du cours. Puis on a lancé `docker images`. L'image pèse 475 Mo sur disque. Son contenu compressé

pèse 131 Mo.

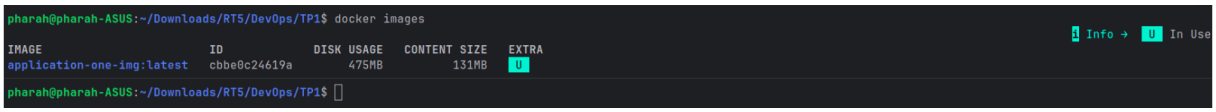


FIGURE 1 – Image de l'application Spring Boot du cours

Ensuite, on a ajouté un identifiant Docker Hub devant le nom de l'image avec `docker tag`. L'image `ayebfarah/application-one-img` garde le même identifiant que l'image locale.

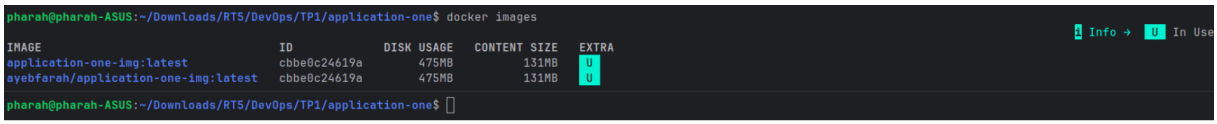


FIGURE 2 – Ajout du tag Docker Hub

Cette image sert de repère. Notre image Python sera presque cinq fois plus légère.

3 Partie B, étape 0 : environnement

On démarre d'abord Docker Desktop. Sans lui, le client Docker ne trouve pas le moteur. Puis on vérifie l'installation et on se connecte au registre.

```
docker version
docker login
mkdir TP_Docker
cd TP_Docker
```

Listing 1 – Préparation de l'environnement

La commande `docker version` doit afficher le client et le serveur. Les deux apparaissent dans la capture de la partie Jenkins.

4 Partie B, étape 1 : Dockerfile et isolation

4.1 L'application Flask

L'application contient quatre éléments.

- Elle se connecte au serveur Redis nommé `db-service`.
- Elle incrémente la clé `hits` à chaque visite.
- Elle affiche le compteur et l'identifiant du conteneur.
- Elle renvoie une erreur 503 si Redis ne répond pas.

Elle expose aussi la route `/health`. Cette route sert aux tests.

```
import os
import socket

import redis
from flask import Flask

app = Flask(__name__)

REDIS_HOST = os.getenv("REDIS_HOST", "db-service")
REDIS_PORT = int(os.getenv("REDIS_PORT", "6379"))


def get_redis():
    return redis.Redis(
        host=REDIS_HOST,
        port=REDIS_PORT,
        socket_connect_timeout=2,
        decode_responses=True,
    )


@app.route("/")
def index():
    try:
        hits = get_redis().incr("hits")
    except redis.exceptions.RedisError:
        return "Erreur : impossible de joindre Redis (db-service).\n", 503
    container_id = socket.gethostname()
    return (
        f"Bonjour ! Cette page a été vue {hits} fois. "
        f"Je suis le conteneur {container_id}\n"
    )


@app.route("/health")
def health():
    return "ok\n", 200


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Listing 2 – Fichier app.py

Le fichier des dépendances fixe les versions.

```
flask==3.0.3
redis==5.0.8
gunicorn==22.0.0
```

Listing 3 – Fichier requirements.txt

4.2 Les tests unitaires

Les tests utilisent un faux Redis. Ils ne demandent donc aucun serveur. Ils vérifient trois points. Le compteur augmente. La page contient l'identifiant du conteneur. La route de santé répond.

```
import app as app_module


class FakeRedis:
    def __init__(self):
        self.store = {}

    def incr(self, key):
        self.store[key] = self.store.get(key, 0) + 1
        return self.store[key]


def make_client(monkeypatch):
    fake = FakeRedis()
    monkeypatch.setattr(app_module, "get_redis", lambda: fake)
    return app_module.app.test_client()


def test_counter_increments(monkeypatch):
    client = make_client(monkeypatch)
    r1 = client.get("/")
    r2 = client.get("/")
    assert r1.status_code == 200
    assert "vue 1 fois" in r1.get_data(as_text=True)
    assert "vue 2 fois" in r2.get_data(as_text=True)


def test_page_contains_container_id(monkeypatch):
    client = make_client(monkeypatch)
    body = client.get("/").get_data(as_text=True)
    assert "On suis le conteneur" in body


def test_health(monkeypatch):
    client = make_client(monkeypatch)
    assert client.get("/health").status_code == 200
```

Listing 4 – Fichier tests/test_app.py

On lance les tests avec `python -m pytest -v`. Les trois tests réussissent.

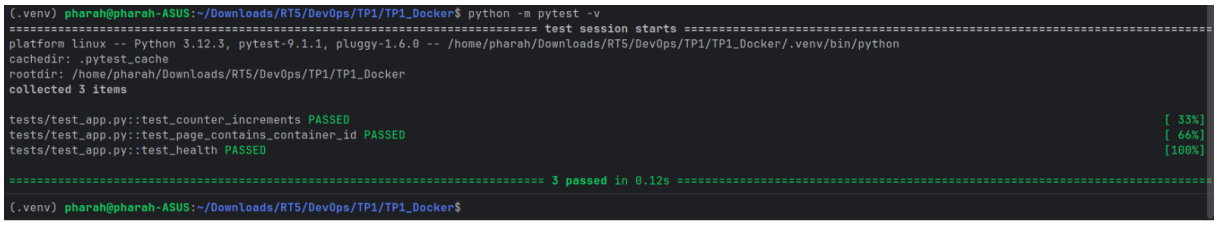


FIGURE 3 – Les trois tests unitaires réussisse

4.3 Le Dockerfile commenté

```
# Image de base : python:3.12-alpine
# Justification : Alpine (~50 Mo) est bien plus légère que l'image complète
# (~1 Go). Nos dépendances (flask, redis-py, gunicorn) sont du Python pur :
# pas de compilation, donc pas de problème de compatibilité musl.
FROM python:3.12-alpine

# Pas de .pyc inutiles, logs affichés immédiatement
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

# 1) On copie d'abord SEULEMENT requirements.txt :
#    tant que les dépendances ne changent pas, cette couche (et le pip install)
#    reste en cache Docker, même si app.py change.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 2) Puis le code source (couche qui change souvent, donc en dernier)
COPY app.py .

# Utilisateur non-root : limite l'impact d'une compromission
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

EXPOSE 5000

# CMD : commande exécutée au démarrage du conteneur (forme exec, sans shell)
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
```

Listing 5 – Fichier Dockerfile

4.4 Justification des choix

Image de base : On utilise `python:3.12-alpine`. Alpine pèse environ 50 Mo. L'image Python complète pèse environ 1 Go. Nos dépendances sont du Python pur. Elles ne demandent aucune compilation. Alpine convient donc très bien.

Utilisateur sans privilèges : L'instruction `USER appuser` évite de lancer l'application en root. Une attaque sur l'application donne alors moins de droits.

Cache Docker : On copie d'abord `requirements.txt`. On installe les dépendances. On copie le code ensuite. Si On modifie seulement `app.py`, Docker réutilise la couche des dépendances. Le build devient très rapide.

Fichier .dockerignore : Il empêche de copier des fichiers inutiles dans l'image, comme `.git`, les tests et les fichiers de cache.

```
__pycache__/
*.pyc
.git
.env
tests/
Jenkinsfile
docker-compose.yml
*.md
venv/
```

Listing 6 – Fichier .dockerignore

4.5 Build et vérifications

```
docker build -t counter-app:1.0 .
docker images counter-app
docker run --rm counter-app:1.0 id
```

Listing 7 – Commandes de la partie 1

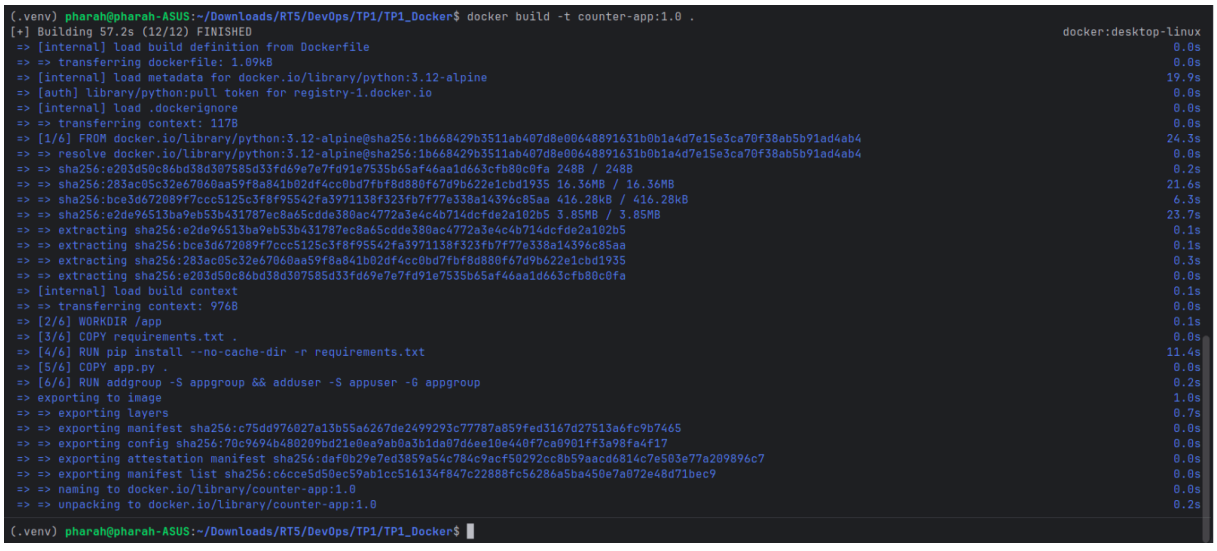


FIGURE 4 – Construction de l'image counter-app :1.0

Notre premier build donne une image de 114 Mo sur disque. Cette image contenait pytest dans `requirements.txt`. On a retiré pytest. Le second build donne une image de 96,3 Mo sur disque. Son contenu compressé pèse 23,5 Mo.

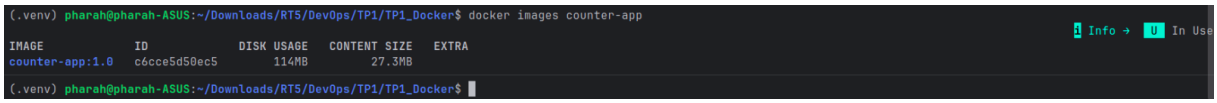


FIGURE 5 – Premier build : 114 Mo



FIGURE 6 – Image finale : 96,3 Mo

La commande `id` prouve que l'application ne tourne pas en root. Elle affiche l'utilisateur `appuser` avec l'identifiant 100.

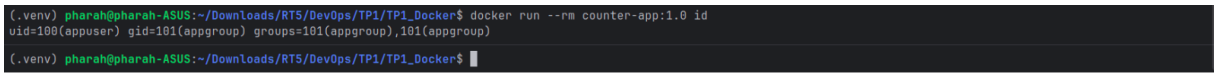


FIGURE 7 – L'utilisateur du conteneur n'est pas root

5 Partie B, étape 2 : réseau et volume

Dans cette partie, On relie les conteneurs à la main.

5.1 Création du réseau et du volume

```
docker network create --driver bridge app-network
docker volume create redis-data
```

Listing 8 – Réseau et volume

Le réseau `app-network` permet aux conteneurs de se joindre par leur nom. Le volume `redis-data` garde les données de Redis.

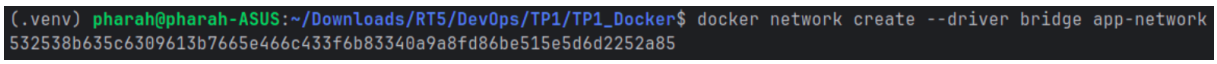


FIGURE 8 – Création du réseau app-network



FIGURE 9 – Création du volume redis-data

5.2 Lancement de Redis

```
docker run -d \
  --name db-service \
  --network app-network \
  -v redis-data:/data \
  redis:7-alpine redis-server --appendonly yes
```

Listing 9 – Lancement de Redis

On nomme le conteneur `db-service`. L'application utilise ce nom pour joindre Redis. On monte le volume sur `/data`.

L'option `--appendonly yes` est importante. Par défaut, Redis sauvegarde par instantanés à intervalles. La commande `docker rm -f` tue le conteneur sans sauvegarde finale. Le compteur peut alors disparaître. Le mode AOF écrit chaque opération dans `/data`. Le volume garde donc le compteur.

```
(.venv) pharah@pharah-ASUS:~/Downloads/RT5/DevOps/TP1/TP1_Docker$ docker run -d \
  --name db-service \
  --network app-network \
  -v redis-data:/data \
  redis:7-alpine redis-server --appendonly yes
Unable to find image 'redis:7-alpine' locally
7-alpine: Pulling from library/redis
4eb14bae0e1d: Pull complete
9928009b15c8: Pull complete
16333ee0c00f: Pull complete
1522f9c6f453: Pull complete
4f4fb700ef54: Pull complete
7eb50503c2b0: Pull complete
83234986c742: Pull complete
54779914f171: Download complete
3beebc06c4ca: Download complete
Digest: sha256:858f009f9709ce576febc734aa78b8f6d624b82571f9ddb6bda4377c833b3499
Status: Downloaded newer image for redis:7-alpine
5b1a81d5549eef108c3f4cb2a8bd9c4b825b68d95c6d241c0e2d70d7f5a8e727
```

FIGURE 10 – Lancement de Redis

```
(.venv) pharah@pharah-ASUS:~/Downloads/RT5/DevOps/TP1/TP1_Docker$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS          PORTS      NAMES
5b1a81d5549e   redis:7-alpine   "docker-entrypoint.s…"   About a minute ago   Up About a minute   6379/tcp   db-service
```

FIGURE 11 – Redis en cours d'exécution

5.3 Lancement de l'application

```
docker run -d \
  --name visit-app \
  --network app-network \
  -p 80:5000 \
  counter-app:1.0
```

Listing 10 – Lancement de l'application

Le port 80 de l'hôte mène au port 5000 de l'application.

```
(.venv) pharah@pharah-ASUS:~/Downloads/RT5/DevOps/TP1/TP1_Docker$ docker run -d \
  --name visit-app \
  --network app-network \
  -p 80:5000 \
  counter-app:1.0
00cb6391db3a8e79506d350b99e40b392ba67319482497c3dafb3dc103eed0ef
```

FIGURE 12 – Lancement de l'application

5.4 Test du compteur

On teste avec curl. Le compteur monte à chaque requête. L'identifiant du conteneur reste le même.

```
pharah@pharah-ASUS:~/Downloads/RT5/DevOps/DevOps/Conteneurisation et Orchestration Micro-services/TP1_Docker$ curl http://localhost
Bonjour ! Cette page a été vue 1 fois. Je suis le conteneur 158fe8533225
pharah@pharah-ASUS:~/Downloads/RT5/DevOps/DevOps/Conteneurisation et Orchestration Micro-services/TP1_Docker$ curl http://localhost
Bonjour ! Cette page a été vue 2 fois. Je suis le conteneur 158fe8533225
pharah@pharah-ASUS:~/Downloads/RT5/DevOps/DevOps/Conteneurisation et Orchestration Micro-services/TP1_Docker$ curl http://localhost
Bonjour ! Cette page a été vue 3 fois. Je suis le conteneur 158fe8533225
pharah@pharah-ASUS:~/Downloads/RT5/DevOps/DevOps/Conteneurisation et Orchestration Micro-services/TP1_Docker$ curl http://localhost
Bonjour ! Cette page a été vue 4 fois. Je suis le conteneur 158fe8533225
pharah@pharah-ASUS:~/Downloads/RT5/DevOps/DevOps/Conteneurisation et Orchestration Micro-services/TP1_Docker$ curl http://localhost
Bonjour ! Cette page a été vue 5 fois. Je suis le conteneur 158fe8533225
```

FIGURE 13 – Test du compteur avec curl

On teste aussi dans le navigateur.

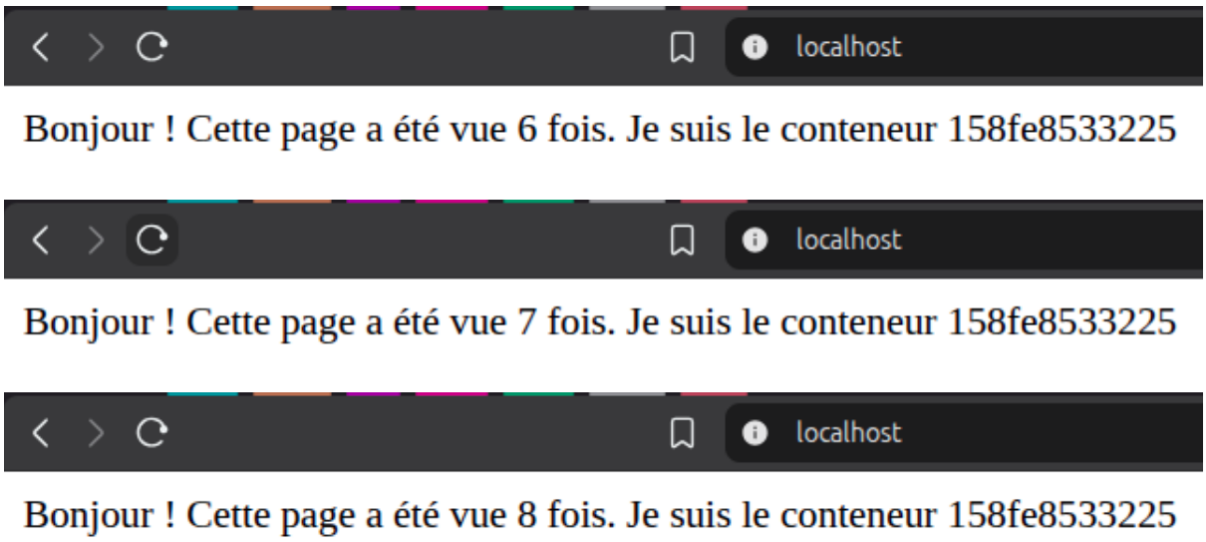


FIGURE 14 – L'application dans le navigateur

5.5 Le réseau

La commande `docker network inspect app-network` liste les conteneurs connectés. On vois `visit-app` et `db-service`.

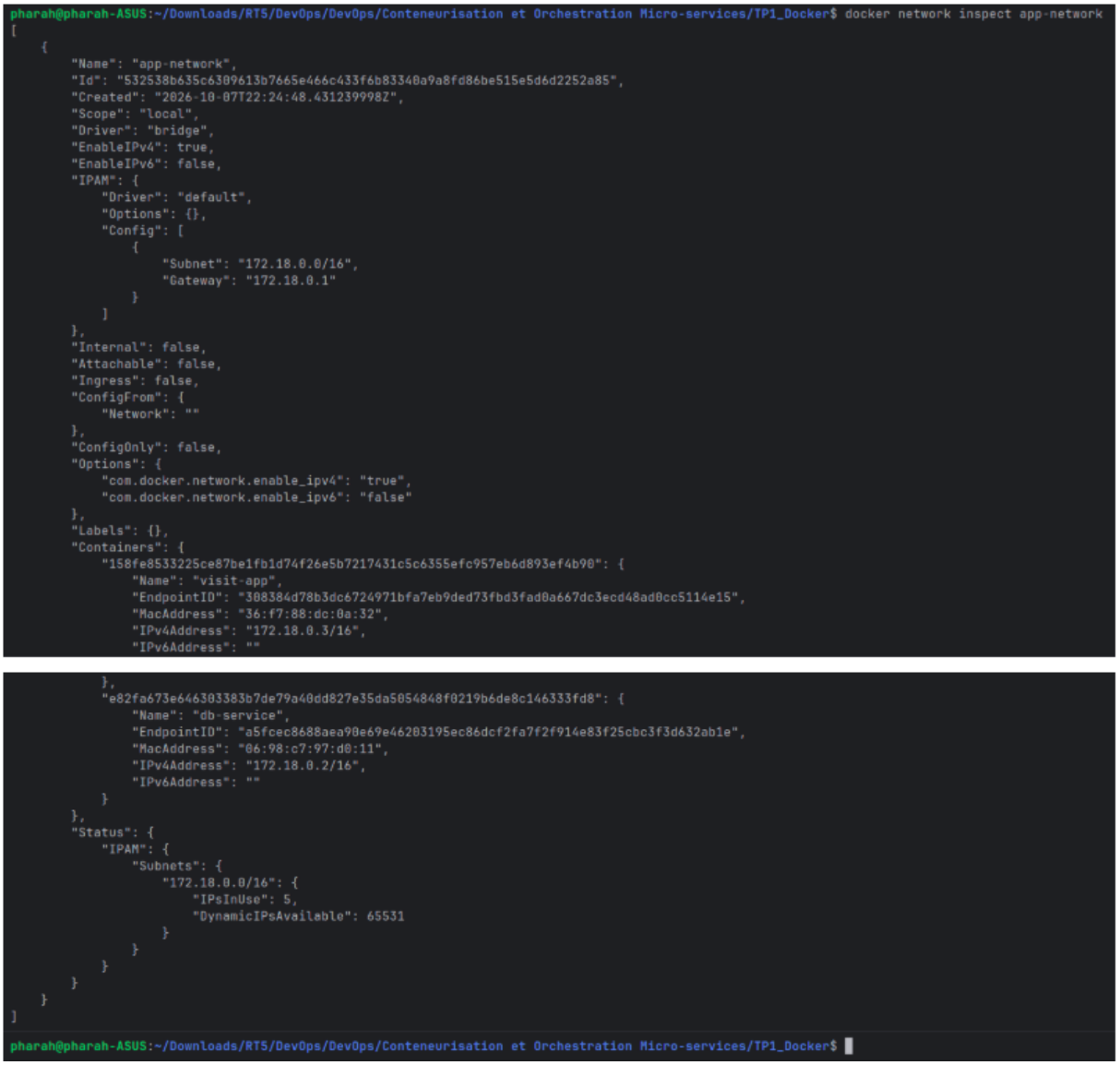


FIGURE 15 – Les conteneurs connectés à app-network

5.6 Test de robustesse

On supprime Redis avec `docker rm -f db-service`. On le relance avec la même commande et le même volume. Le compteur passe de 9 à 10. Il garde donc sa valeur.



FIGURE 16 – Le compteur survit à la suppression de Redis

6 Partie B, étape 3 : Docker Compose

6.1 Les fichiers de configuration

Le fichier `.env` contient les ports et les images. On ne met aucun port en dur dans le fichier YAML.

```
WEB_HOST_PORTS=9080-9090
WEB_CONTAINER_PORT=5000
LB_PORT=8088
REDIS_PORT=6379
REDIS_IMAGE=redis:7-alpine
APP_IMAGE=counter-app:1.0
```

Listing 11 – Fichier .env

Nos avons utilisé les ports 9080 à 9090 et le port 8088. Jenkins occupe le port 8081

```
services:
  db-service:
    image: ${REDIS_IMAGE}
    command: ["redis-server", "--appendonly", "yes"]
    volumes:
      - redis-data:/data
    networks:
      - app-network
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5
    restart: unless-stopped

  web:
    image: ${APP_IMAGE}
    build: .
    # No container_name: required for scaling.
    # Port range: each instance takes the next free host port.
    ports:
      - "${WEB_HOST_PORTS}:${WEB_CONTAINER_PORT}"
    environment:
```

```
      REDIS_HOST: db-service
      REDIS_PORT: ${REDIS_PORT}
    depends_on:
      db-service:
        condition: service_healthy
    networks:
      - app-network
    restart: unless-stopped

  lb:
    image: nginx:alpine
    ports:
      - "${LB_PORT}:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - web
    networks:
      - app-network
    restart: unless-stopped

networks:
  app-network:
    external: true

volumes:
  redis-data:
    name: redis-data
```

Listing 12 – Fichier docker-compose.yml final

Voici les choix importants.

- Le service web n'a pas de `container_name`. Sans nom fixe, Compose peut créer plusieurs instances.
- Le service web publie une plage de ports. Chaque instance prend le prochain port libre.
- Le nom du service Redis est `db-service`. Ce nom devient son nom DNS.
- Le healthcheck attend que Redis réponde avant de lancer le web.
- Le réseau `app-network` est externe. On l'ai créé à la main dans la partie 2.

6.2 Lancement avec trois instances

```
docker compose up -d --build --scale web=3
docker compose ps
```

Listing 13 – Lancement du scaling

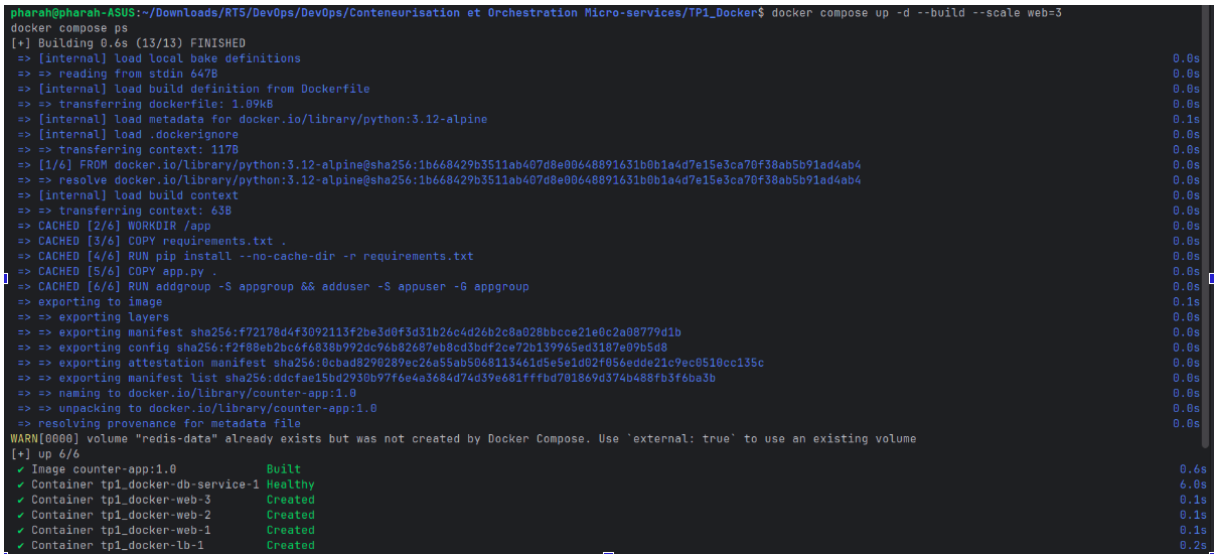


FIGURE 17 – Lancement avec trois instances web



FIGURE 18 – Les services du déploiement Compose

6.3 Test des instances

On teste chaque instance sur son port. On obtient trois identifiants différents. Le compteur continue d'augmenter. Les trois instances partagent donc le même Redis.

```
for p in 9080 9081 9082
do
  curl -s http://localhost:$p
done
```

Listing 14 – Test des trois instances

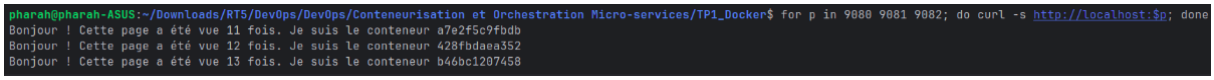


FIGURE 19 – Trois instances, trois identifiants, un seul compteur

6.4 Analyse : pourquoi pas le même port hôte ?

Un port de l'hôte accepte un seul conteneur à la fois. Imaginons que les trois instances demandent le port 80. La première le prend. Les deux autres échouent avec l'erreur `port is already allocated`.

Chaque conteneur a son propre espace réseau. Le port interne 5000 ne pose donc aucun conflit. Seul le port publié sur l'hôte doit être unique.

6.5 Solution : un load balancer

Un load balancer règle ce problème. Il publie seul un port sur l'hôte. Les trois instances web ne publient plus rien. Elles restent joignables dans `app-network`. Le load balancer reçoit les requêtes et les répartit entre les instances. Il utilise le nom DNS `web` fourni par Docker.

```
events {}
http {
  resolver 127.0.0.11 valid=5s;      # DNS interne de Docker
  server {
    listen 80;
    location / {
      set $upstream http://web:5000; # variable => re-résolution DNS régulière
      proxy_pass $upstream;
    }
  }
}
```

Listing 15 – Fichier nginx.conf

L'adresse 127.0.0.11 est le DNS interne de Docker. Il renvoie les adresses des trois instances. La variable `$upstream` force Nginx à interroger le DNS toutes les cinq secondes.

Dans nos tests, les identifiants changent d'une requête à l'autre. La répartition n'est pas parfaitement régulière. Elle suit les réponses du DNS.

```
(.venv) pharah@pharah-ASUS:~/Downloads/RT5/DevOps/TP1/TP1_Docker$ curl http://localhost
Bonjour ! Cette page a été vue 11 fois. Je suis le conteneur eea2ec76c3d5

(.venv) pharah@pharah-ASUS:~/Downloads/RT5/DevOps/TP1/TP1_Docker$ curl http://localhost
Bonjour ! Cette page a été vue 12 fois. Je suis le conteneur 50db7311c136

(.venv) pharah@pharah-ASUS:~/Downloads/RT5/DevOps/TP1/TP1_Docker$ curl http://localhost
Bonjour ! Cette page a été vue 13 fois. Je suis le conteneur eea2ec76c3d5

(.venv) pharah@pharah-ASUS:~/Downloads/RT5/DevOps/TP1/TP1_Docker$ curl http://localhost
Bonjour ! Cette page a été vue 14 fois. Je suis le conteneur dabbb52048b4

(.venv) pharah@pharah-ASUS:~/Downloads/RT5/DevOps/TP1/TP1_Docker$ curl http://localhost
Bonjour ! Cette page a été vue 15 fois. Je suis le conteneur 50db7311c136

(.venv) pharah@pharah-ASUS:~/Downloads/RT5/DevOps/TP1/TP1_Docker$ curl http://localhost
Bonjour ! Cette page a été vue 16 fois. Je suis le conteneur dabbb52048b4
```

FIGURE 20 – Le load balancer répartit les requêtes



FIGURE 21 – Test du load balancer sur le port 8088

6.6 Test de persistance avec Compose

On arrête puis on relance toute la pile. Le compteur passe de 20 à 21. Le volume garde donc les données.

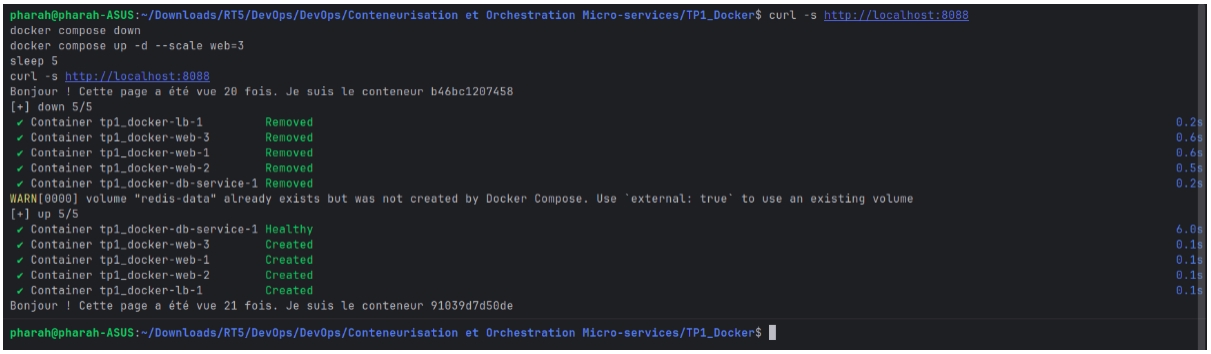


FIGURE 22 – Le compteur survit à un arrêt complet de Compose

7 Partie B, étape 4 : publication et nettoyage

7.1 Publication sur Docker Hub

On tague l'image avec notre identifiant Docker Hub. On pousse deux tags : `1.0` et `latest`.

```
docker tag counter-app:1.0 ayebfarah/counter-app:1.0
docker tag counter-app:1.0 ayebfarah/counter-app:latest
docker push ayebfarah/counter-app:1.0
docker push ayebfarah/counter-app:latest
```

Listing 16 – Publication de l'image

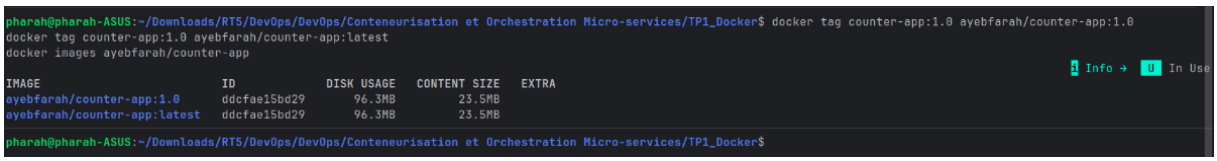


FIGURE 23 – Capture des commandes `docker tag` et `docker images`

```
pharah@pharah-ASUS:~/Downloads/RT5/DevOps/DevOps/Conteneurisation et Orchestration Micro-services/TP1_Docker$ docker push ayebfarah/counter-app:1.0
docker push ayebfarah/counter-app:latest
The push refers to repository [docker.io/ayebfarah/counter-app]
0d7a9ce41b69: Pushed
25b9e81e040b: Pushed
bce3d672089f: Layer already exists
d1db767f315a: Pushed
e203d50c86bd: Layer already exists
11ea5eefdf48: Pushed
e2de96513ba9: Layer already exists
283ac05c32e6: Layer already exists
e6e0c773c0b6: Pushed
be0da7fe6c90: Pushed
1.0: digest: sha256:ddcfae15bd2930b97f6e4a3684d74d39e681fffbd701869d374b488fb3f6ba3b size: 856
The push refers to repository [docker.io/ayebfarah/counter-app]
0d7a9ce41b69: Layer already exists
e6e0c773c0b6: Layer already exists
e203d50c86bd: Layer already exists
bce3d672089f: Layer already exists
d1db767f315a: Layer already exists
e2de96513ba9: Layer already exists
be0da7fe6c90: Layer already exists
11ea5eefdf48: Already exists
25b9e81e040b: Layer already exists
283ac05c32e6: Layer already exists
latest: digest: sha256:ddcfae15bd2930b97f6e4a3684d74d39e681fffbd701869d374b488fb3f6ba3b size: 856
```

FIGURE 24 – Envoi de l'image sur Docker Hub

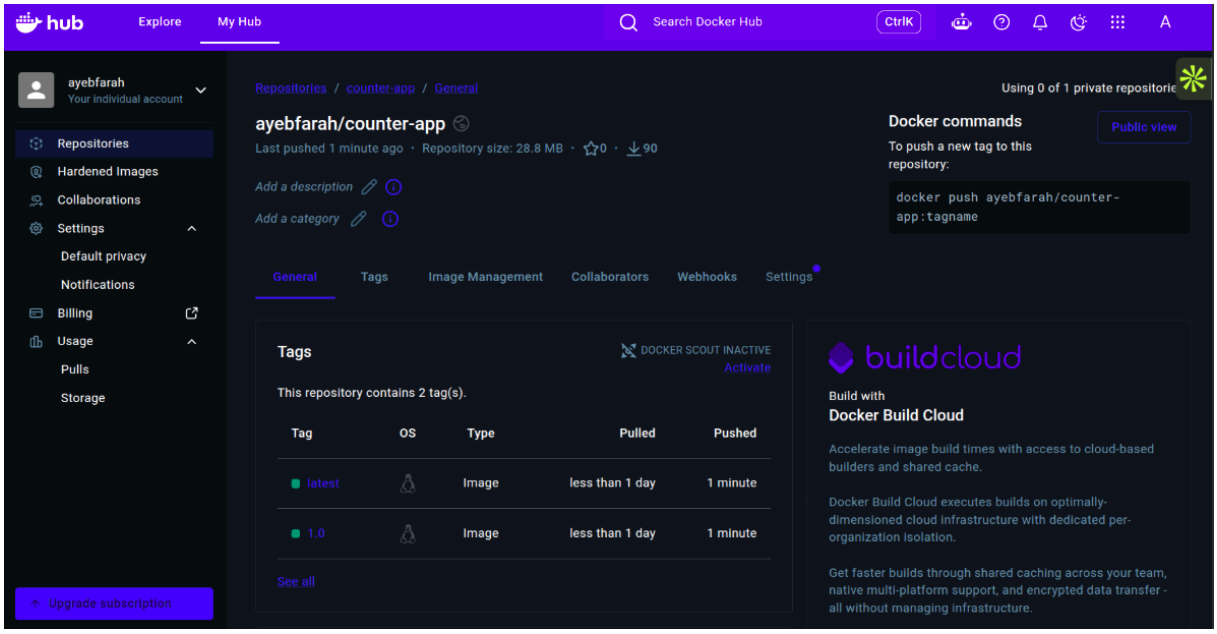


FIGURE 25 – Dépôt Docker Hub avec les tags 1.0 et latest

Le dépôt est public. Voici le lien :

`https://hub.docker.com/r/ayebfarah/counter-app`

7.2 Nettoyage en profondeur

Une seule commande supprime tout ce qui est inutilisé.

```
docker system prune -a --volumes -f
```

Listing 17 – Nettoyage en profondeur

Elle supprime les conteneurs arrêtés, les réseaux inutilisés, les images inutilisées, le cache de build et les volumes inutilisés. Elle touche tout Docker, pas seulement ce TP. On l'exécute donc à la fin.

8 Réflexion : RUN et CMD

	RUN	CMD
Moment	Pendant le build de l'image	Au démarrage du conteneur
Effet	Crée une nouvelle couche dans l'image	Définit la commande par défaut. Aucune couche créée.
Nombre	Autant que nécessaire	Une seule effective, la dernière
Remplaçable	Non, le résultat est figé dans l'image	Oui, avec `docker run image autre commande`

Table 2 – Différences entre RUN et CMD

Voici un exemple :
`RUN pip install -r requirements.txt` installe les dépendances une fois dans l'image.
`CMD ["gunicorn", ...]` lance l'application à chaque démarrage du conteneur.

9 Partie C : pipeline Jenkins

Le pipeline démarre à chaque commit sur la branche principale. Il exécute les tests. Il construit l'image. Il la publie sur Docker Hub.

9.1 Installation de Jenkins

On lance Jenkins dans un conteneur. On monte le socket Docker. Jenkins peut ainsi utiliser le moteur Docker de ma machine.

```
docker run -d --name jenkins \
  -p 8081:8080 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -u root \
  jenkins/jenkins:lts-jdk17
```

Listing 18 – Lancement de Jenkins

On utilise le port 8081. Le volume `jenkins_home` garde la configuration de Jenkins.

On installe ensuite le client Docker et Python dans le conteneur Jenkins.

```
docker exec -it jenkins sh -c "apt-get update && apt-get install -y docker.io
    python3-venv python3-pip"
docker exec jenkins docker version
```

Listing 19 – Outils dans le conteneur Jenkins

La commande `docker version` doit afficher le client et le serveur.

```
pharah@pharah-ASUS:~/Downloads/RT5/DevOps/DevOps/Conteneurisation et Orchestration Micro-services/TP1_Docker$ docker exec jenkins docker version
Client:
 Version:           26.1.5+dfsg1
 API version:       1.45
 Go version:        go1.24.4
 Git commit:        a72d7cd
 Built:             Thu Aug 13 05:35:47 2026
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Desktop 4.61.0 (219004)
 Engine:
  Version:          29.2.1
  API version:      1.53 (minimum version 1.44)
  Go version:       go1.25.6
  Git commit:       6bc6209
  Built:            Mon Feb  2 17:17:24 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.2.1
  GitCommit:        dea7da592f5d1d2b7755e3a161be07f43fad8f75
 runc:
  Version:          1.3.4
  GitCommit:        v1.3.4-0-gd6d73eb8
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

FIGURE 26 – Lancement de Jenkins et test de Docker

9.2 Identifiants Docker Hub

On a créé un jeton d'accès sur Docker Hub. On l'a enregistré dans Jenkins avec l'identifiant `dockerhub-credentials`. Le mot de passe ne figure donc jamais dans le code.

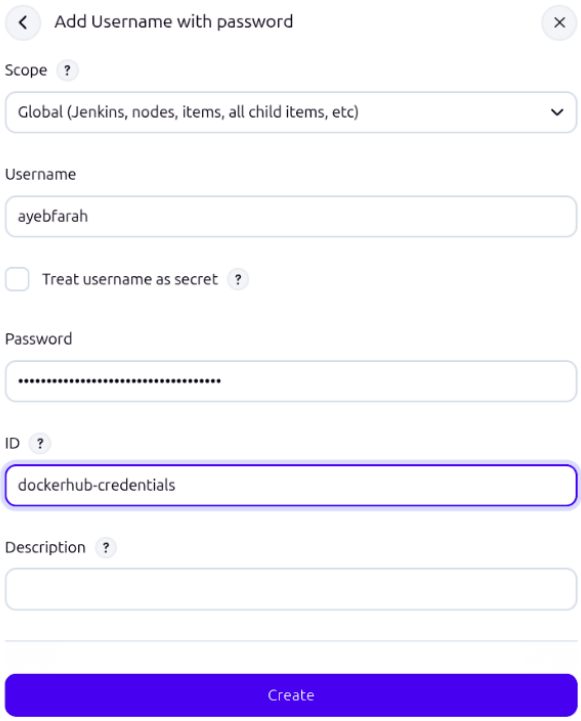


FIGURE 27 – Création de l'identifiant dans Jenkins

9.3 Création du job

On crée un job de type Pipeline nommé `visit-counter`. On règle les paramètres suivants.

— Définition : Pipeline script from SCM.
— SCM : Git, avec l'adresse du dépôt GitHub.
— Branche : `*/main`.
— Chemin du script : le Jenkinsfile dans le dossier TP1_Docker.

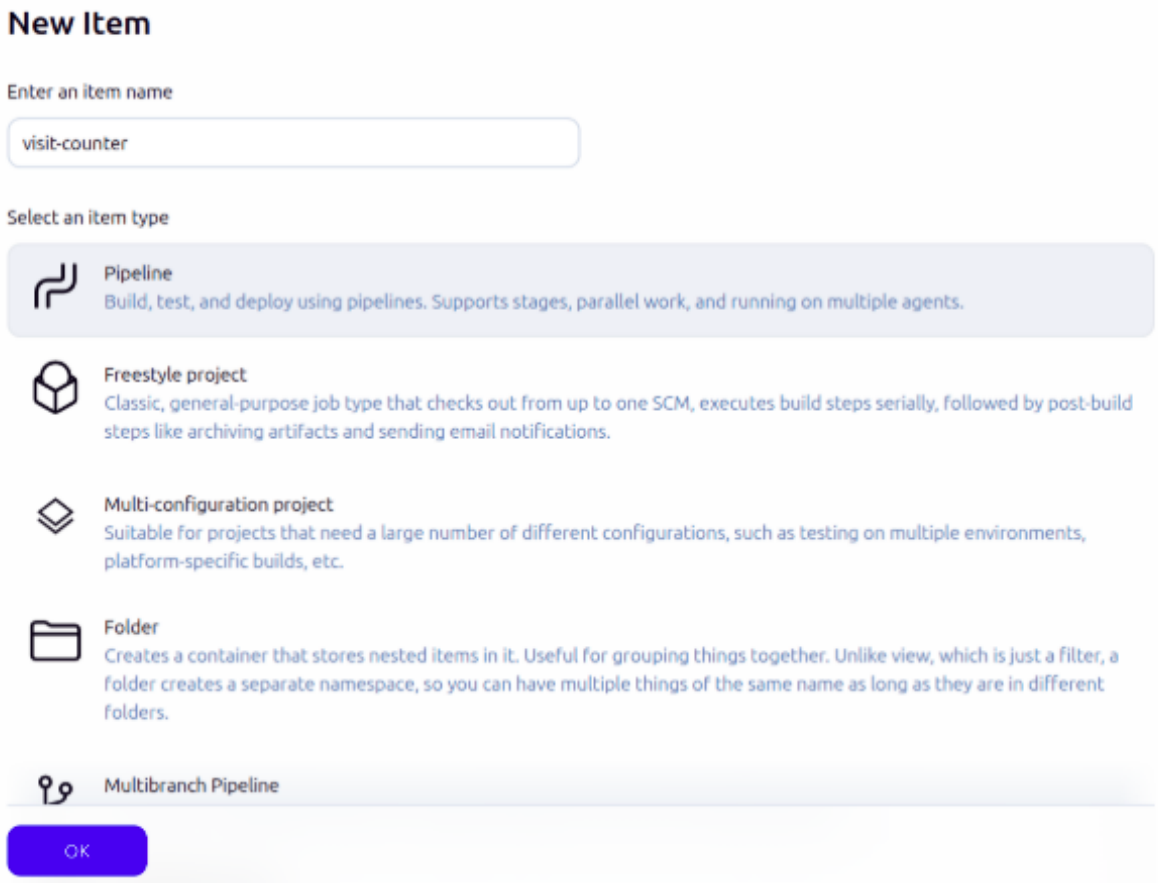


FIGURE 28 – Création du job visit-counter

9.4 Le Jenkinsfile

Le TP se trouve dans un dossier du repo partagé. La variable `APP_DIR` indique ce dossier. Chaque étape s'exécute dedans.

```
pipeline {
    agent any

    environment {
        DOCKERHUB_USER = 'ayebfarah'
        IMAGE_NAME     = "${DOCKERHUB_USER}/counter-app"
        IMAGE_TAG      = "${env.BUILD_NUMBER}"
        APP_DIR        = 'Conteneurisation et Orchestration Micro-services/
   TP1_Docker'
```

```
    }

    triggers {
        githubPush()
        pollSCM('* * * * *')
    }

    options {
        timestamps()
        buildDiscarder(logRotator(numToKeepStr: '10'))
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build & Tests unitaires') {
            steps {
                dir("${APP_DIR}") {
                    sh '''
                        python3 -m venv venv
                        . venv/bin/activate
                        python -m pip install --no-cache-dir -r requirements.
txt pytest
                        python -m pytest -v --junitxml=test-results.xml
                    '''
                }
            }
            post {
                always {
                    junit allowEmptyResults: true, testResults: "${APP_DIR}/
test-results.xml"
                }
            }
        }

        stage('Build image Docker') {
            when {
                anyOf {
                    branch 'main'
                    expression { env.GIT_BRANCH == 'origin/main' }
                }
            }
            steps {
                dir("${APP_DIR}") {
                    sh 'docker build -t ${IMAGE_NAME}:${IMAGE_TAG} -t ${
IMAGE_NAME}:latest .'
                }
```

```
            }
        }

        stage('Publication Docker Hub') {
            when {
                anyOf {
                    branch 'main'
                    expression { env.GIT_BRANCH == 'origin/main' }
                }
            }
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'dockerhub-credentials',
                    usernameVariable: 'DH_USER',
                    passwordVariable: 'DH_PASS')]) {
                    sh '''
                        echo "$DH_PASS" | docker login -u "$DH_USER" --password
    -stdin
                        docker push ${IMAGE_NAME}:${IMAGE_TAG}
                        docker push ${IMAGE_NAME}:latest
                    '''
                }
            }
        }
    }

    post {
        always {
            sh 'docker logout || true'
        }
        success {
            echo "Image publiee : ${IMAGE_NAME}:${IMAGE_TAG}"
        }
        failure {
            echo 'Pipeline en echec : consulter les logs.'
        }
    }
}
```

Listing 20 – Fichier Jenkinsfile

Le pipeline compte quatre étapes.

1. **Checkout** récupère le code depuis GitHub.
2. **Build et tests unitaires** crée un environnement Python, installe les dépendances et lance pytest. Si un test échoue, le pipeline s'arrête.
3. **Build image Docker** construit l'image. Elle reçoit le numéro du build et le tag `latest`. Cette étape tourne seulement sur la branche principale.
4. **Publication Docker Hub** se connecte avec le jeton et pousse les deux tags.

Déclenchement automatique : Jenkins tourne sur mon poste. GitHub ne peut pas l'appeler. On utilise donc `pollSCM`. Jenkins interroge GitHub chaque minute. Il lance un

build dès qu'il voit un nouveau commit. Avec une adresse publique, un webhook GitHub ferait le même travail plus vite.

9.5 Résultats

On a créé un nouveau jeton avec les droits de lecture et d'écriture. On a mis à jour l'identifiant dans Jenkins. Le build a réussi. Docker Hub affiche alors le tag 2 à côté de `latest` et de `1.0`.

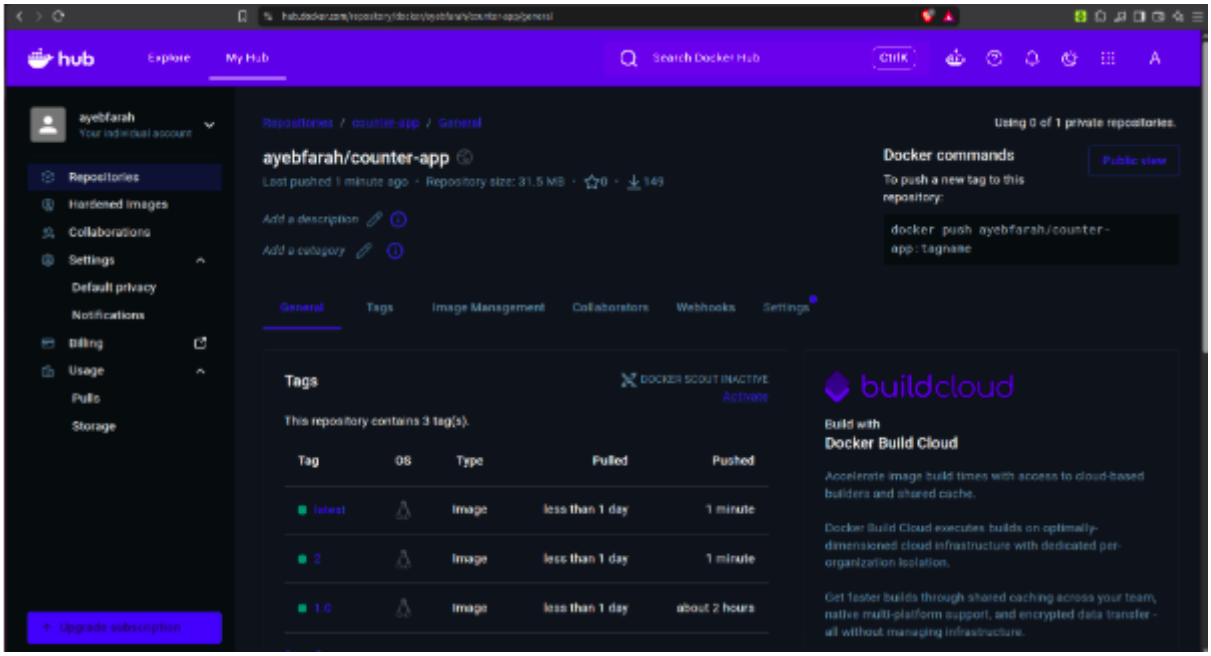


FIGURE 29 – Docker Hub après le build

9.6 Test du déclenchement automatique

On modifie le texte dans `app.py`. On fais un commit. On pousse sur la branche principale. On n'appuie sur aucun bouton dans Jenkins. Environ une minute plus tard, le build 3 démarre seul. Il réussit.

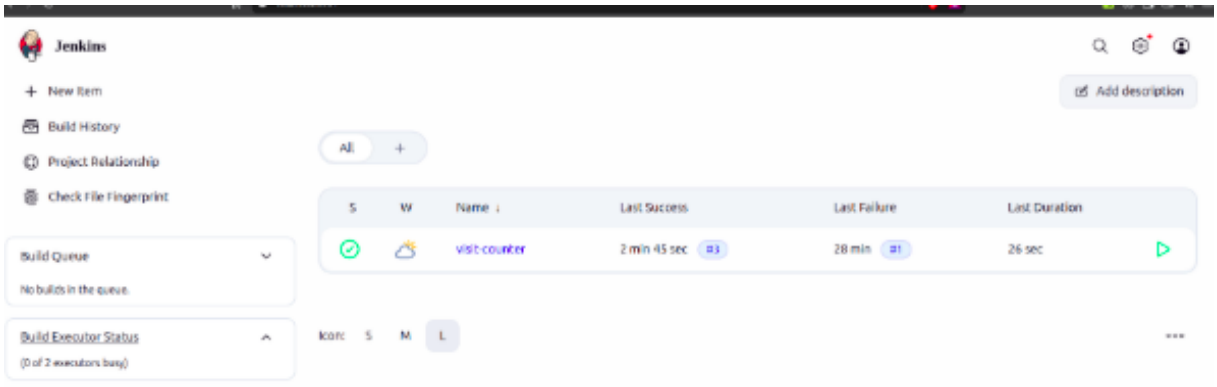


FIGURE 30 – Le build 3 démarre après le push

Docker Hub affiche le nouveau tag 3.

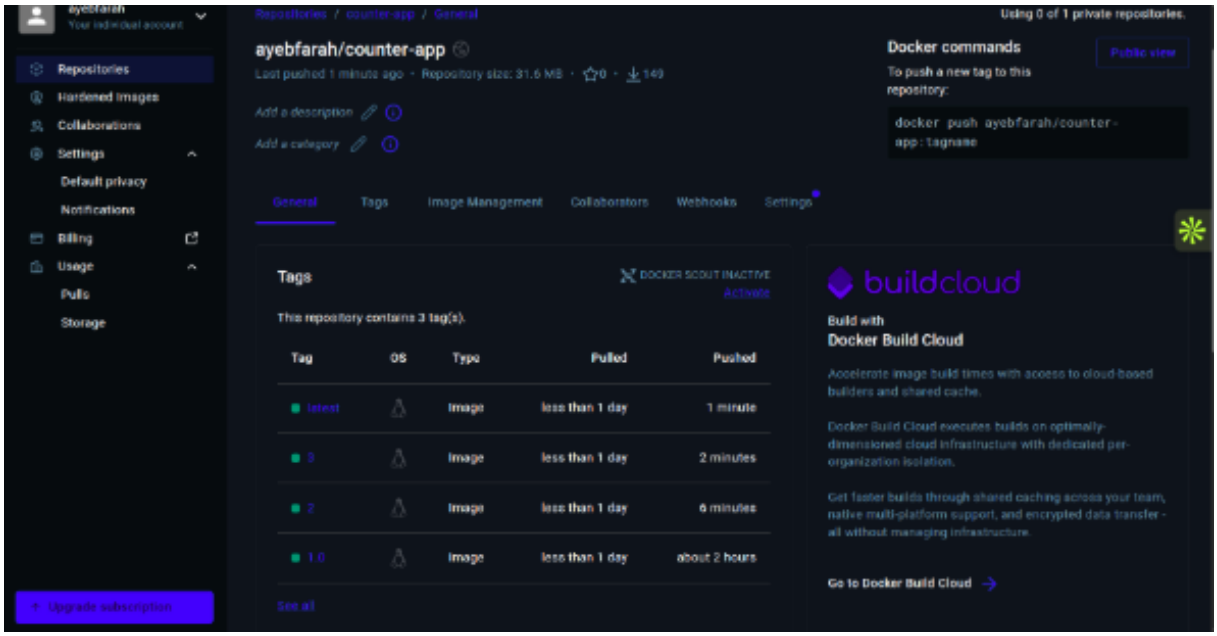


FIGURE 31 – Docker Hub après le build 3

Liens du projet

— Dépôt GitHub : `https://github.com/AyebFarah/DevOps`
— Image Docker Hub : `https://hub.docker.com/r/ayebfarah/counter-app`